Cinema FAW

Gestión de Películas Full-Stack

React 19 · Flask 2.3 · TMDB API · SQLite

Alberto Jiménez

Descripción del Proyecto

Propósito

Desarrollar aplicación web completa para catalogar y gestionar películas integrando fuentes locales y externas.

Stack Tecnológico

- React 19.1.1 + TypeScript 5.9.3 (Frontend)
- Flask 2.3.3 + SQLite (Backend)
- TMDB API v3 (Fuente externa)
- Bootstrap 5.3.8 (Estilos)

Características Clave

- Catálogo local (CRUD completo)
- Búsqueda en TMDB e importación
- Sistema de Favoritos sincronizado
- Eliminación de películas
- Interfaz responsive con Bootstrap 5

Sistema dividido en tres capas

1 — FRONTEND (React)

Interfaz responsiva con hooks, gestión de estado y comunicación REST

2 — BACKEND (Flask)

API REST con endpoints CRUD, persistencia en SQLite y validación de datos

3 — INTEGRACIÓN (TMDB)

Búsqueda de películas, mapeo de datos e importación al catálogo

Stack Tecnológico

🔷 FRONTEND

- React 19.1.1
- TypeScript 5.9.3
- Vite 7.1.12
- Bootstrap 5.3.8
- React Router 7

Puerto: 5175

↔ HTTP REST

🔶 BACKEND

- Flask 2.3.3
- Flask-CORS
- SQLite 3
- Python 3.x

Puerto: 5000

→ Fetch Data

🎬 TMDB API v3

- Movie Search Endpoint
- Popular Movies Endpoint
- Movie Details Endpoint
- Genre List Endpoint

Proveedor: The Movie DB

Arquitectura: Patrón ViewModel con Custom Hooks

Estructura de Carpetas del Proyecto

```
src/
  hooks/  (Custom Hooks = ViewModels)
    useMoviesViewModel.ts
    useMovieDetailViewModel.ts
    useSearchTMDBViewModel.ts
  pages/  (Vistas que consumen ViewModels)
    MoviesPage.tsx
    MovieDetailPage.tsx
    SearchTMDBPage.tsx
  services/  (Lógica de negocio)
    tmdbService.ts
    api.ts
    favoritesStore.ts
  components/  (Componentes reutilizables)
    MovieCard.tsx
    MovieList.tsx
```

¿Qué es ViewModel?

El patrón ViewModel separa la lógica de negocio de la presentación:

Ventajas

- Código más testeable
- Reutilización de lógica
- Componentes más simples
- Estados centralizados

Implementación

- Custom Hooks en React
- Hooks retornan estado + métodos
- Páginas consumen los Hooks
- Componentes son presentacionales

Flujo de Datos y Sincronización de Estado

📄 **Component (React Page)**

🔧 **ViewModel (Custom Hook)**

⚙️ **Services (API/Store)**

💾 **Data Sources (Backend/Store)**

1. Página Carga
Usuario abre /movies → Componente se monta → Llama useMoviesViewModel()

2. Obtiene Datos
ViewModel ejecuta GET /api/movies → Backend retorna películas → Estado se actualiza

3. Sincroniza Favoritos
favoritesStore carga favoritos de localStorage → Combina con películas → UI se renderiza

4. Estado Listo
Componente muestra datos + favoritos → Usuario puede interactuar (eliminar, marcar favorito)

Sincronización de Favoritos (localStorage)

MoviesPage ↔ read/write **favoritesStore** ↔ sync **MovieDetailPage**

✅ No se requiere HTTP para favoritos
✅ localStorage es compartido entre ventanas del navegador
✅ Cambios instantáneos sin delay de red

REST API — Endpoints Implementados

Método	Endpoint	Descripción	Status	Consumido Por
GET	`/api/movies`	Obtener todas las películas del catálogo	**200 OK**	`useMoviesViewModel`
GET	`/api/movies/:id`	Obtener película por su ID	**200 OK**	`useMovieDetailViewModel`
POST	`/api/movies`	Crear nueva película en catálogo local	**201 Created**	`useSearchTMDBViewModel`
DELETE	`/api/movies/:id`	Eliminar película del catálogo	**204 No Content**	`useMoviesViewModel`

📌 Todos los endpoints están configurados en backend/app.py
📌 CORS habilitado para http://localhost:5173 (frontend React)
📌 Las respuestas son JSON (application/json)
📌 SQLite usa UNIQUE(title, year) para evitar duplicados

Flujo típico: Frontend envía → Flask valida → SQLite persiste → Response JSON retorna

Pantalla 1: Catálogo Local

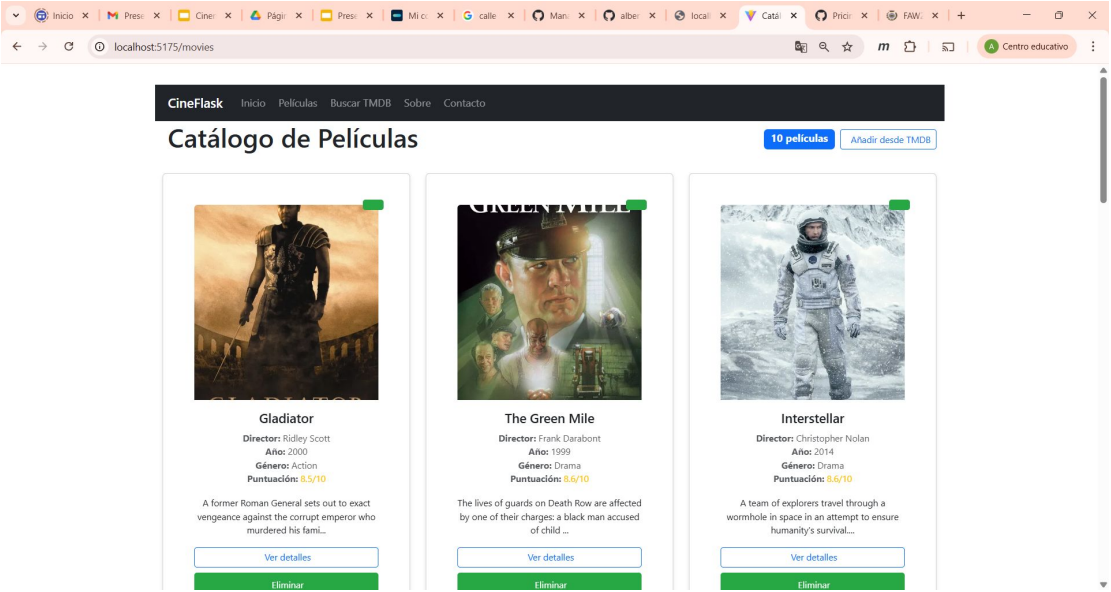
Pantalla principal (/movies) que muestra todas las películas del catálogo local.

- Películas cargadas desde GET /api/movies
- Datos persistidos en SQLite (backend)
- Botón eliminar para remover película
- Interfaz responsive con Bootstrap 5

Header con navegación

Grid de películas (MovieCard)

Botón eliminar



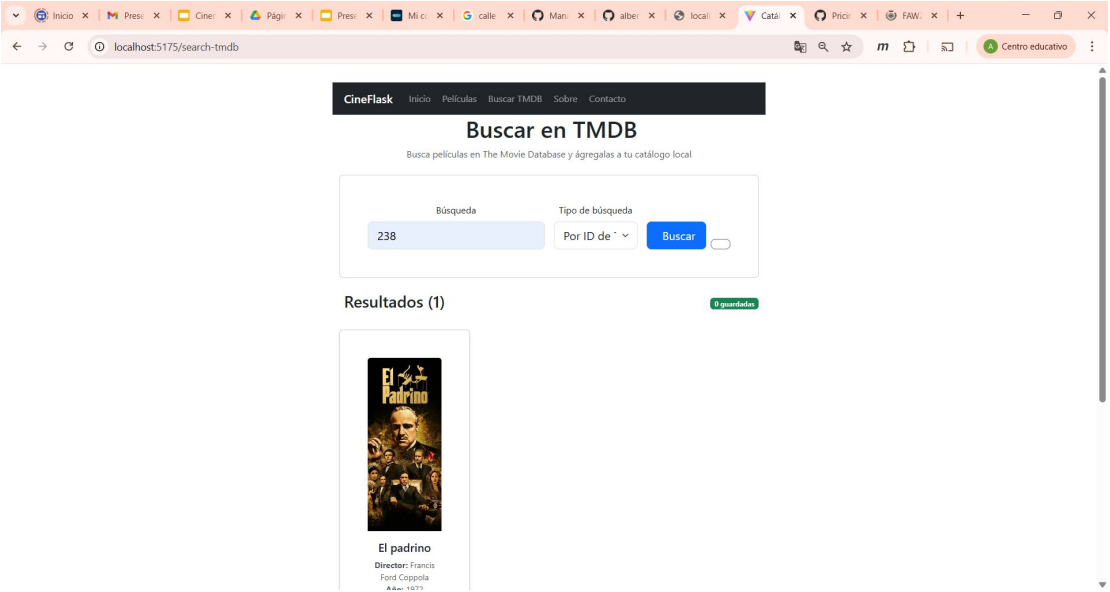
Pantalla 2: Búsqueda e Importación TMDB

Página de búsqueda (/search-tmdb) con integración TMDB API v3.

- Búsqueda por ID de película (búsqueda directa)
- Búsqueda por texto (búsqueda fuzzy)
- Datos reales desde The Movie Database

Barra de búsqueda (por ID o texto)

Resultados de TMDB API



Pantalla 3: Detalle de Película

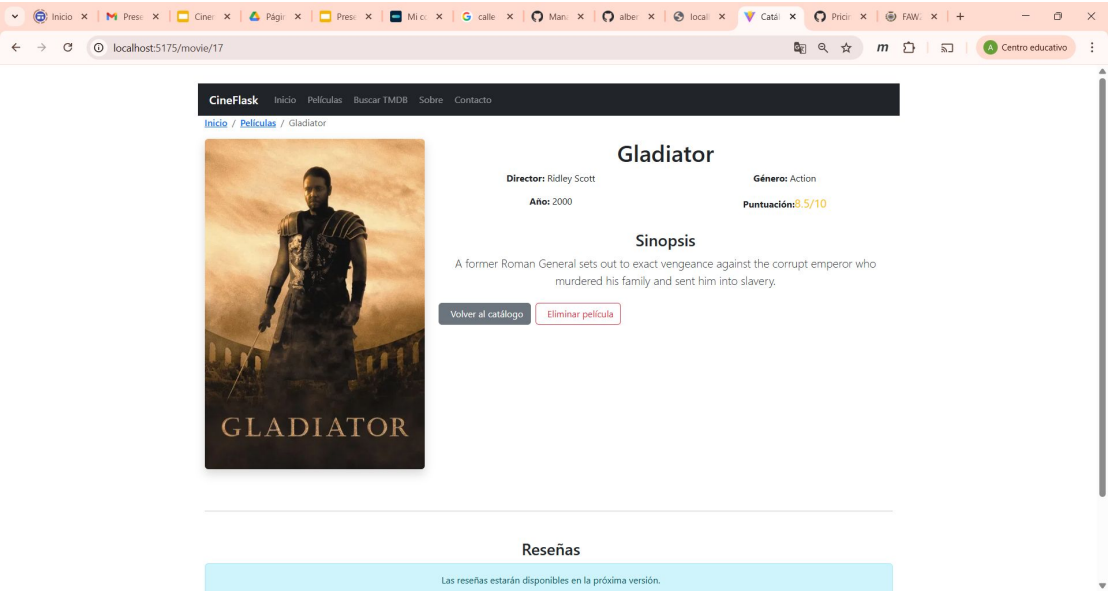
Vista de detalle (/movies/:id) con información completa de la película.

- Información completa: título, director, año, sinopsis
- Botón eliminar película del catálogo
- Favoritos sincronizados con localStorage
- Navegación de regreso al catálogo

Poster de película

Info: GET /api/movies/:id

Botón eliminar



Base de Datos — Estructura SQLite

Archivo proyecto.db ubicado en backend/proyecto.db con dos tablas principales:

Tabla MOVIES

```
CREATE TABLE movies (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  title TEXT NOT NULL,
  director TEXT,
  year INTEGER,
  genre TEXT,
  description TEXT,
  rating REAL DEFAULT 0.0,
  poster_url TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE(title, year)
)
```

Tabla CONTACTS

```
CREATE TABLE contacts (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  email TEXT NOT NULL,
  subject TEXT,
  message TEXT NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)
```

🔒 Constraint UNIQUE(title, year)

Previene la inserción de películas duplicadas al nivel de la base de datos.

❌ Sin constraint: Error histórico que causó 41 registros duplicados de 2 películas.

✅ Con constraint: INSERT OR IGNORE automáticamente evita duplicados.

✅ Dos películas con mismo title pero diferente año son permitidas.

✅ Integridad de datos garantizada sin validación manual en la aplicación.

Retos Técnicos y Soluciones

1. Sincronización de ViewModels

Desafío	Causa	Solución
Mantener estado consistente entre MoviesPage, MovieDetailPage y SearchTMDBPage. Favoritos cambiaban en una página sin reflejarse en otra.	Cada Hook mantenía su propio estado local. Componentes no compartían información entre sí.	Crear favoritesStore.ts centralizado. localStorage único. Todos los Hooks leen/escriben el mismo store. Cambios instantáneos.

2. Deduplicación de Películas

Síntoma	Raíz del Problema	Solución
41 registros duplicados: The Shawshank Redemption (20 copias) y The Godfather (21 copias).	Sin UNIQUE constraint. Ruta relativa 'proyecto.db' creaba múltiples archivos. INSERT sin validación en BD.	UNIQUE(title, year) en CREATE TABLE. Ruta absoluta con os.path.abspath(). Eliminar BD antigua y reiniciar.

3. Integración Frontend–Backend (CORS)

Problema	Causa	Solución
Error: 'Access to XMLHttpRequest blocked by CORS policy'. Frontend en 5175, Backend en 5000.	Política del mismo origen del navegador. Flask no declaraba orígenes permitidos para fetch() desde React.	pip install flask-cors. CORS(app, origins=['http://localhost:5173']). Comunicación REST fluida.

Conclusiones y Aprendizajes

Logros Clave

✓ **Full-Stack Completo**
React moderno + Flask robusto integrados

✓ **Patrones Arquitectónicos**
ViewModel con Custom Hooks, separación de responsabilidades

✓ **Integración API Externa**
TMDB integrado, búsqueda e importación funcional

✓ **Sincronización de Estado**
favoritesStore centralizado sin sobrecarga del servidor

✓ **Base de Datos Robusta**
SQLite con constraints, integridad de datos garantizada

Lecciones Aprendidas

🎯 **Constraints en BD**
UNIQUE constraints previenen horas de debugging de duplicados

🎯 **Rutas Absolutas**
os.path.abspath() en lugar de rutas relativas resuelve portabilidad

🎯 **Centralizar Estado**
favoritesStore como única fuente de verdad simplifica la sincronización

🎯 **CORS en Producción**
Configurar correctamente desde el inicio ahorra frustración

🎯 **Testing Manual**
Validar cada feature en tiempo de desarrollo previene bugs tardíos

Ejecución Local y Repositorio

Terminal 1 — Backend (Flask)

```
# cd Backend\Flask-backend\backend
python app.py

# Puerto: 5000
# Verificar: localhost:5000/api/movies
```

Terminal 2 — Frontend (React)

```
# cd FAW26_REACT_Terminado
npm run dev

# Puerto: 5175
# URL: http://localhost:5175
```

Ruta	URL	Descripción
Catálogo	`localhost:5175/movies`	Lista de películas desde BD local
Detalle	`localhost:5175/movies/:id`	Detalles de película específica
Búsqueda TMDB	`localhost:5175/search-tmdb`	Buscar e importar desde TMDB
Contacto	`localhost:5175/contact`	Formulario de contacto

📍 **Repositorio GitHub:** `https://github.com/albertojh9/FAW26_REACT_Terminado`
📁 Frontend: FAW26_REACT_Terminado · Backend: Backend/Flask-backend/backend

📄 Clave: src/hooks/ (ViewModels) · backend/app.py (API) · backend/models.py (DB)